

TP1 — Définir une stratégie de branching et de promotion réseau

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

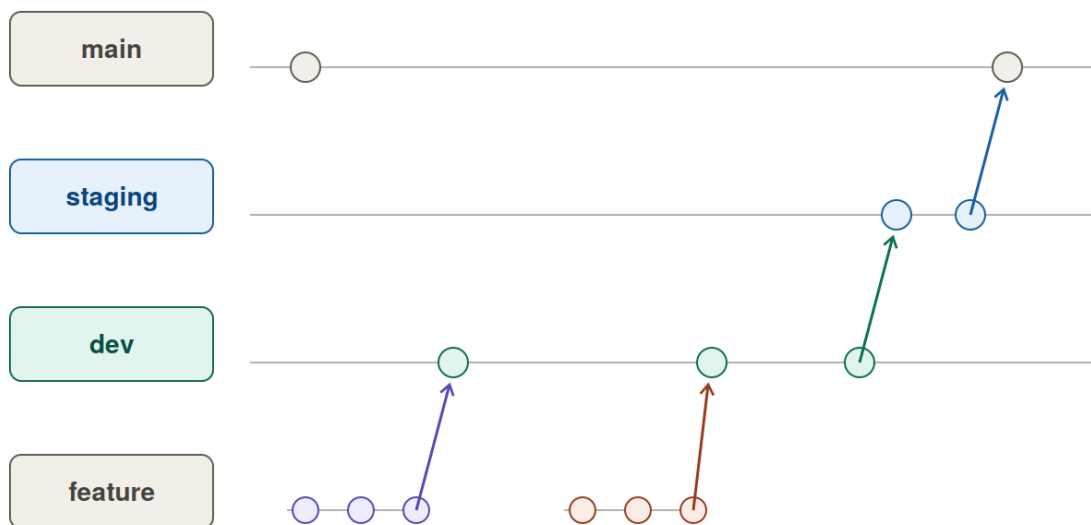


Figure 1: Modèle de branches NexaCorp

Ce TP met en place le dépôt Git partagé du projet fil rouge NexaCorp et la stratégie de branches qui sera utilisée pendant toute la formation : **main** / **staging** / **dev** en branches permanentes, et une branche `<prenom>-<nom>-jourX-tpN` par étudiant et par jour. Le TP se termine par une mise en pratique en groupe (pull request, revue croisée, résolution de conflit) pour vivre le workflow avant de l'utiliser au quotidien.

Prérequis

- Un ordinateur avec droits d'installation de logiciels
- Un compte GitHub personnel (gratuit) — <https://github.com>
- Connaissances de base en ligne de commande

Étapes

Étape 0 — Mise en place du dépôt partagé (formateur, une seule fois)

1. Créer un dépôt sur GitHub : nom netops-project-<nom-groupe>, visibilité Private, avec README
2. Inviter les étudiants : **Settings > Collaborators and teams > Add people**, rôle **Write**
3. Protéger les branches main et staging : **Settings > Branches > Add branch ruleset** → cocher **Require a pull request before merging** et **Require approvals** (1 minimum)
4. Créer l'environnement protégé production : **Settings > Environments > New environment** → cocher **Required reviewers**

Étape 1 — Installer Git

Windows :

```
# Télécharger et installer depuis https://git-scm.com/download/win
# (installe aussi Git Bash, à utiliser pour toutes les commandes de ce TP)
git --version
```

macOS :

```
brew install git
```

Linux (Debian/Ubuntu) :

```
sudo apt update
sudo apt install git -y
```

Sous Windows, ouvrez **Git Bash** (clic droit dans le dossier → *Git Bash Here*) :
mkdir -p et touch n'existent pas nativement en PowerShell.

Étape 2 — Configurer son identité Git

```
git config --global user.name "Votre Nom"
git config --global user.email "vous@exemple.com"
```

Étape 3 — Cloner le dépôt partagé

```
git clone https://github.com/<formateur>/netops-project-<nom-groupe>.git
cd netops-project-<nom-groupe>
pwd
```

Étape 4 — Créer la structure de dossiers du projet

```
mkdir -p inventory/group_vars roles templates vault tests/pre_checks tests/post_checks
touch inventory/group_vars/.gitkeep roles/.gitkeep templates/.gitkeep vault/.gitkeep
ls -R
```

Étape 5 — Créer les branches permanentes

```
git checkout -b dev
git push -u origin dev

git checkout main
git checkout -b staging
git push -u origin staging

git checkout dev
```

Étape 6 — Documenter la stratégie de branching

```
cat > docs/branching-strategy.md << 'EOF'
# Strategie de branching NetOps - NexaCorp

## Branches permanentes
- main      : configuration en production sur les sites reels (protegee)
- staging    : configuration validee en lab, en attente de validation finale (protegee)
- dev       : branche d'integration commune du groupe

## Branches temporaires
- <prenom>-<nom>-jourX-tpN : une branche par etudiant, par jour de formation

## Workflow quotidien
1. git checkout dev && git pull origin dev
2. git checkout -b <prenom>-<nom>-jourX-tpN
3. Travailler et committer sur cette branche
4. git push -u origin <prenom>-<nom>-jourX-tpN
5. Ouvrir une pull request <prenom>-<nom>-jourX-tpN -> dev
6. Apres revue, merger dans dev

## Regle de promotion
1. PR dev -> staging : declenche les tests automatises
2. PR staging -> main : declenche le deploiement en production
EOF

git add .
git commit -m "Initialisation structure projet + strategie de branching"
git push origin dev
```

Étape 7 — Créer sa branche du jour

```
git checkout -b <prenom>-<nom>-jour1-tp1
git push -u origin <prenom>-<nom>-jour1-tp1
```

Étape 8 — Mise en pratique : expérimenter le workflow en groupe

Tous les étudiants modifient le même fichier, chacun sur sa branche, pour provoquer une vraie pull request et un conflit.

```
git checkout <prenom>-<nom>-jour1-tp1
git pull origin dev --no-rebase
```

Créer/éditer inventory/group_vars/all.yml :

```
organisation: nexacorp
environnement: dev
sites:
  - nom: siege
    role: hub
  - nom: agence1
    role: spoke
  - nom: agence2
    role: spoke
contributeurs:
  - prenom: <votre-prenom>
    role: netops-student
```

```
git add .
git commit -m "Ajout de <votre-prenom> aux contributeurs"
git push -u origin <prenom>-<nom>-jour1-tp1
```

Sur GitHub : ouvrir une pull request <prenom>-<nom>-jour1-tp1 → dev. Un camarade (assigné par le formateur) fait une revue (commentaire + **Approve**).

Les pull requests sont mergées **une par une**. En cas de conflit, Git l'indique dans le fichier :

```
<<<<<< HEAD
- prenom: marie
  role: netops-student
=====
- prenom: karim
  role: netops-student
>>>>>> dev
```

Garder les deux lignes, supprimer les balises <<<<<< / ===== / >>>>>>, puis :

```
git add inventory/group_vars/all.yml
git commit -m "Resolution conflit contributeurs"
git push origin <prenom>-<nom>-jour1-tp1
```

Une fois tout le groupe mergé dans dev : ouvrir une PR dev → staging, puis staging → main (approbation requise sur main).

Validation

- ☐ `git --version` fonctionne dans Git Bash
- ☐ Le dépôt partagé existe sur GitHub, tous les étudiants l'ont cloné
- ☐ Les branches `dev`, `staging`, `main` existent et sont poussées sur GitHub
- ☐ `main` et `staging` sont protégées contre le push direct
- ☐ `docs/branching-strategy.md` est présent et décrit le workflow
- ☐ Chaque étudiant a sa branche `<prenom>-<nom>-jour1-tp1` poussée sur GitHub
- ☐ Chaque étudiant a ouvert une pull request vers `dev` et reçu une revue d'un pair
- ☐ Au moins un conflit de merge a été rencontré et résolu manuellement
- ☐ La chaîne `dev` → `staging` → `main` a été exécutée par le groupe